```
# ==============================
# Data Handling Libraries
# ==============================
import pandas as pd          # For handling tabular data
import numpy as np           # For numerical operations

# ==============================
# Visualization Libraries
# ==============================
import matplotlib.pyplot as plt   # For plotting graphs
import seaborn as sns             # For advanced visualizations

# ==============================
# Hugging Face Libraries
# ==============================
from datasets import load_dataset     # To load dataset from Hugging Face
from transformers import (
    BertTokenizer,                    # Tokenizer for BERT
    BertForSequenceClassification,    # BERT model for classification
    Trainer,                          # Trainer API
    TrainingArguments                 # Training configuration
)

# ==============================
# Evaluation Libraries
# ==============================
from sklearn.metrics import (
    accuracy_score,
    precision_recall_fscore_support,
    confusion_matrix,
    classification_report
)

import torch   # Deep learning backend (PyTorch)
```

```
dataset = load_dataset("sms_spam")

print(dataset)
print(dataset["train"][0])
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:        4.98k/?  [00:00<00:00,  351kB/s]

plain_text/train-00000-of-00001.parquet:  100%                                                    359k/359k  [00:00<00:00,  1.79MB/s]

Generating train split:  100%                                                    5574/5574  [00:00<00:00,  79958.72  examples/s]

DatasetDict({
    train: Dataset({
        features: ['sms', 'label'],
        num_rows: 5574
    })
})
{'sms': 'Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...\n', 'la
```

```
# Load pre-trained BERT tokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

# Tokenization function
def tokenize_function(example):
    return tokenizer(
        example["sms"],
        padding="max_length",   # Pad shorter sequences
        truncation=True,        # Truncate long sequences
        max_length=128
    )

# Apply tokenization
tokenized_dataset = dataset.map(tokenize_function, batched=True)
```

```
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 3.65kB/s]

vocab.txt: 232k/? [00:00<00:00, 2.90MB/s]

tokenizer.json: 466k/? [00:00<00:00, 3.51MB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en

Map: 100%  5574/5574 [00:01<00:00, 5573.81 examples/s]
```

```
tokenized_dataset = tokenized_dataset.rename_column("label", "labels")

tokenized_dataset.set_format(
    "torch",
    columns=["input_ids", "attention_mask", "labels"]
)
```

```
train_test = tokenized_dataset["train"].train_test_split(test_size=0.2)

train_dataset = train_test["train"]
test_dataset = train_test["test"]
```

```
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
config.json: 100%  570/570 [00:00<00:00, 45.2kB/s]

model.safetensors: 100%  440M/440M [00:04<00:00, 287MB/s]

Loading weights: 100%  199/199 [00:00<00:00, 503.47it/s, Materializing param=bert.pooler.dense.weight]

BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
--------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
cls.seq_relationship.weight                 | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |
cls.predictions.transform.dense.bias        | UNEXPECTED |
cls.seq_relationship.bias                   | UNEXPECTED |
cls.predictions.transform.dense.weight      | UNEXPECTED |
cls.predictions.bias                        | UNEXPECTED |
classifier.bias                             | MISSING    |
classifier.weight                           | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

```
training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    logging_dir="./logs"
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` instead.
```

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    accuracy = accuracy_score(labels, predictions)
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average='binary', zero_division=0
    )
    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1,
```

```
    }

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)

trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true bu
  super().__init__(loader)
```

[281/837 1:33:10 < 3:05:39, 0.05 it/s, Epoch 1.00/3]

Step	Training Loss

[837/837 4:33:13, Epoch 3/3]

Step	Training Loss
500	0.044480

Writing model shards: 100% 1/1 [00:07<00:00, 7.65s/it]

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true bu
  super().__init__(loader)
```

Writing model shards: 100% 1/1 [00:10<00:00, 10.78s/it]

```
TrainOutput(global_step=837, training_loss=0.028893701776666312, metrics={'train_runtime': 16415.3663,
'train_samples_per_second': 0.815, 'train_steps_per_second': 0.051, 'total_flos': 879909146887680.0, 'train_loss':
0.028893701776666312, 'epoch': 3.0})
```

```
results = trainer.evaluate()
print(results)
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set as true bu
  super().__init__(loader)
```

[70/70 14:08]

```
{'eval_loss': 0.058882348239421844, 'eval_accuracy': 0.989237668161435, 'eval_precision': 0.9751552795031055, 'eval_recall': 0.9
```